

```
'''
```

```
By      :- Darshan Shrestha
```

```
Roll no.:- KCE081BCT010
```

```
'''
```

```
TOLERABLE_ERROR = 10**-5
```

```
def func(x):
```

```
    return x**2 - 5 * x + 5
```

```
def print_iteration_table(rows) -> None:
```

```
    columns = ["Iteration", "a", "b", "m", "f(a)", "f(m)", "Error"]
```

```
    widths = {
```

```
        "Iteration": len("Iteration"),
```

```
        "a": len("a"),
```

```
        "b": len("b"),
```

```
        "m": len("m"),
```

```
        "f(a)": len("f(a)"),
```

```
        "f(m)": len("f(m)"),
```

```
        "Error": len("Error"),
```

```
    }
```

```
    for row in rows:
```

```
        widths["Iteration"] = max(widths["Iteration"],  
len(f"{row['Iteration']}"))
```

```
        for key in ("a", "b", "m", "f(a)", "f(m)", "Error"):
```

```
            widths[key] = max(widths[key], len(f"{row[key]:.6f}"))
```

```
    header = " | ".join(f"{column:>{widths[column]}}" for column in columns)
```

```
    separator = "--".join("-" * widths[column] for column in columns)
```

```

print("\nBy      :- Darshan Shrestha")
print("Roll no. :- KCE081BCT010")
print("\nIteration Table:\n")
print(header)
print(separator)
for row in rows:
    print(
        f"{row['Iteration']:>{widths['Iteration']}} | "
        f"{row['a']:>{widths['a']}.6f} | "
        f"{row['b']:>{widths['b']}.6f} | "
        f"{row['m']:>{widths['m']}.6f} | "
        f"{row['f(a)']:>{widths['f(a)']}.6f} | "
        f"{row['f(m)']:>{widths['f(m)']}.6f} | "
        f"{row['Error']:>{widths['Error']}.6f}"
    )

```

```

def findRoot(a, b) -> list:
    iterations = 0
    rows = []
    while True:
        m = (a + b) / 2
        func_a = func(a)
        func_m = func(m)
        error = abs(b - a) / 2

        rows.append({
            "Iteration": iterations + 1,
            "a": a,
            "b": b,

```

```

        "m": m,
        "f(a)": func_a,
        "f(m)": func_m,
        "Error": error,
    })

    if func_m * func_a < 0:
        b = m
    else:
        a = m

    iterations += 1
    if abs(func_m) < TOLERABLE_ERROR:
        return [m, iterations, rows]

def main() -> None:
    check = True
    while check:
        print("f(x) = x**2 - 5 * x + 5\n")
        initGuess_1, initGuess_2 = map(float, input("Enter initial guesses"
            "(2 only; separate with spaces).=> ").split())

        if (func(initGuess_1) * func(initGuess_2) >= 0):
            print("Error; the product of the functional value of inputs must
be strictly less than 0.\n")
        else:
            check = False

    a = initGuess_1
    b = initGuess_2

```

```

root, iterations, rows = findRoot(a,b)
print_iteration_table(rows)

root_line = f"Root: {root:.5f}"
iter_line = f"Ran {iterations} iterations"
content_width = max(len(root_line), len(iter_line))
border = "+" + "-" * (content_width + 2) + "+"
print()
print(border)
print(f"| {root_line.ljust(content_width)} |")
print(f"| {iter_line.ljust(content_width)} |")
print(border)

```

```

if __name__ == "__main__":
    main()

```

```

C:\WINDOWS\System32\cmd
f(x) = x**2 - 5 * x + 5
Enter initial guesses(2 only; separate with spaces).=> 1.38 1.39
By :- Darshan Shrestha
Roll no. :- KCE081BCT010
Iteration Table:

```

Iteration	a	b	m	f(a)	f(m)	Error
1	1.380000	1.390000	1.385000	0.004400	-0.006775	0.005000
2	1.380000	1.385000	1.382500	0.004400	-0.001194	0.002500
3	1.380000	1.382500	1.381250	0.004400	0.001602	0.001250
4	1.381250	1.382500	1.381875	0.001602	0.000204	0.000625
5	1.381875	1.382500	1.382187	0.000204	-0.000495	0.000312
6	1.381875	1.382187	1.382031	0.000204	-0.000146	0.000156
7	1.381875	1.382031	1.381953	0.000204	0.000029	0.000078
8	1.381953	1.382031	1.381992	0.000029	-0.000059	0.000039
9	1.381953	1.381992	1.381973	0.000029	-0.000015	0.000020
10	1.381953	1.381973	1.381963	0.000029	0.000007	0.000010

```

+-----+
| Root: 1.38196 |
| Ran 10 iterations |
+-----+
Press any key to continue . . .

```